

1) Dataset Generation

I loaded the Amazon reviews dataset and sampled 50000 of each rating value. I also created a new column "sentiment", setting rows with "star_rating" >3, <3, and =3 to 1, 2, and 3 respectively.

2) Word Embedding

a)

puppy - dog + cat = kitten

horrid ~ dreadful

The analogy example works correctly- puppy is to dog as kitten is to cat, which means that the relative positions of the puppy and dog vectors are similar to those of the kitten and cat vectors. The second example shows that words that are semantically similar have similar vectors.

b)

I performed data cleaning on the review_body by converting to lowercase, removing extra spaces, performing contractions, and removing non-alphabetical characters, then trained a Word2Vec model using embedding size of 300, window size of 11, and min word count of 10.

Using my model, the analogy example yielded "litter" as the result while "horrid" returned the same word "dreadful" as the most similar vector. My model appears to work according to the second example, but the analogy was just slightly off ("litter" is closely related to "kitten"). The pretrained model appears to encode semantic similarities between words better.

3) Simple Models

HW1 perceptron test accuracy: 0.893

HW1 SVM test accuracy: 0.8837

Perceptron test accuracy: 0.831

Perceptron pretrained test accuracy: 0.72135

SVM test accuracy: 0.85165

SVM pretrained test accuracy: 0.83515

For both models, pretrained features resulted in a lower accuracy score compared to my custom features. This suggests that embeddings trained on my own dataset better capture the domain-specific language and vocabulary of the reviews. Pretrained embeddings underperformed likely because they are general-purpose and do not represent slang present in my reviews dataset.

4) Feedforward Neural Networks

a) Accuracy of averaged vectors

NN custom binary test accuracy: 0.871025025844574

NN pretrained binary test accuracy: 0.8593249917030334

NN custom ternary test accuracy: 0.707260012626648

NN pretrained ternary test accuracy: 0.6944800019264221

b) Accuracy of concatenated vectors

NN custom binary (concatenated) test accuracy: 0.7637249827384949

NN pretrained binary (concatenated) test accuracy: 0.7600499987602234

NN custom ternary (concatenated) test accuracy: 0.5982199907302856

NN pretrained ternary (concatenated) test accuracy: 0.5855000019073486

The accuracies of models using pretrained vectors are lower than those using custom vectors. This makes sense as pretrained embeddings are general-purpose and do not capture domain-specific language in the reviews dataset. Additionally, models using concatenated vectors had lower accuracy compared to those using averaged vectors. This is likely due to the concatenated vectors capturing information from only a small portion of each review (first 10 words), which may not contain the necessary signals to distinguish between the sentiments.

5) Convolutional Neural Networks

CNN custom binary test accuracy: 0.880174994468689

CNN pretrained binary test accuracy: 0.7637249827384949

CNN custom ternary test accuracy: 0.7239599823951721

CNN pretrained ternary test accuracy: 0.7201399803161621

The accuracies of the CNN models are higher than those of the feedforward neural network models. This is likely due to the higher number of tokens considered in the CNN example (50 tokens compared to 10 in FFN), as well as the ability of the CNN to encode word order locally and preserve meaningful patterns such as negations.